SOLID · Auth · Tokenizers & Postman

Topics 1–5 | Developer Mastery Guide

SOLID Principles · Coding Standards · Authentication Types LLM Tokenizers · Postman API Practice

TOPIC 1

SOLID Principles (Basic)

SOLID is an acronym for five object-oriented design principles introduced by Robert C. Martin (Uncle Bob). Following them produces code that is easier to maintain, extend, and test.

Letter	Principle
S	Single Responsibility Principle – one class, one job.
O	Open/Closed Principle – open for extension, closed for modification.
L	Liskov Substitution Principle – subtypes must be substitutable for base types.
I	Interface Segregation Principle – no client should depend on methods it doesn't use.
D	Dependency Inversion Principle – depend on abstractions, not concretions.

S — Single Responsibility Principle

A class should have only ONE reason to change.

Every class or function should do exactly one thing. If a class handles both business logic and database access, a change in the DB schema forces a change in business logic code — they should be separate.

Bad Example

```
class User: def get_name(self): ... def save_to_db(self): ... def send_email(self): ... # 3 responsibilities!
```

Good Example

```
class User: # data def get_name(self): ... class UserRepo: # DB def save(self): ... class EmailService: # email def send(self): ...
```

O — Open / Closed Principle

Open for extension, closed for modification.

You should be able to add new functionality without changing existing tested code. Achieve this through inheritance, interfaces, or composition — add new classes rather than editing old ones.

Bad Example

```
def area(shape): if shape == 'circle': return 3.14 * r*r if shape == 'rect': return w * h # Edit this for every new shape!
```

Good Example

```
class Shape: def area(self): ... class Circle(Shape): def area(self): return 3.14*r*r class Rect(Shape): def area(self): return w*h # Add new shapes without editing
```

L Liskov Substitution Principle

Subtypes must be substitutable for their base types.

If class B extends class A, you should be able to replace A with B everywhere without breaking the program. A subclass should honour the contract (behaviour) of its parent — not weaken preconditions or strengthen postconditions.

Bad Example

```
class Bird: def fly(self): ... class Penguin(Bird): def fly(self): raise Exception('Cannot fly') # Breaks LSP!
```

Good Example

```
class Bird: ... class FlyingBird(Bird): def fly(self): ... class Penguin(Bird): ... # Penguin doesn't claim to fly
```

I Interface Segregation Principle

Don't force classes to implement methods they don't use.

Large, general-purpose interfaces should be split into smaller, specific ones. Clients should only know about the methods they actually need. This avoids 'fat interfaces' that bloat implementing classes.

Bad Example

```
class Animal: def eat(self): ... def fly(self): ... def swim(self): ... # Dog must implement fly() and swim()!
```

Good Example

```
class Eater: def eat(self): ... class Flyer: def fly(self): ... class Swimmer: def swim(self): ... class Duck(Eater, Flyer, Swimmer): ...
```

D Dependency Inversion Principle

High-level modules must not depend on low-level modules.

Both high-level and low-level modules should depend on abstractions (interfaces). This decouples your code so you can swap implementations (e.g. switch database) without touching high-level business logic.

Bad Example

```
class OrderService: def __init__(self): self.db = MySQLDatabase() # Tightly coupled to MySQL
```

Good Example

```
class Database: # abstract def save(self): ... class MySQLDB(Database): ... class OrderService: def __init__(self, db: Database): self.db = db # injected
```

Quick Summary

Principle	One-Line Rule
S – SRP	One class, one purpose. Split large classes into focused ones.

O – OCP	Add new features via extension (new classes), not by editing old code.
L – LSP	Subclasses must not break the behaviour expected from the parent class.
I – ISP	Many small interfaces beat one large one. Clients get only what they need.
D – DIP	Inject dependencies. Program to interfaces, not concrete implementations.

TOPIC 2

Coding Standards

Coding standards are agreed-upon rules and conventions that govern how code is written, formatted, and structured within a team or project. They ensure consistency, readability, and maintainability across an entire codebase.

1. Naming Conventions

Element	Convention	Example
Variables	camelCase (JS) / snake_case (Python)	userName / user_name
Constants	UPPER_SNAKE_CASE	MAX_RETRIES = 3
Classes	PascalCase	UserProfile, OrderService
Functions	camelCase / snake_case	getUserById / get_user_by_id
Files	kebab-case (JS) / snake_case (Python)	user-service.js / user_service.py
Booleans	is / has / can prefix	isLoggedIn, hasPermission
Interfaces	I-prefix or Able suffix (C#/Java)	IRepository, Serializable

2. Code Formatting Rules

- Indentation: 2 spaces (JS/TS) or 4 spaces (Python). Never mix tabs and spaces.
- Line length: max 80–120 characters. Use a linter to enforce this.
- Blank lines: 1 blank line between methods; 2 between top-level functions/classes.
- Trailing whitespace: always remove. Configure your editor to do this automatically.
- File ends with newline: a single newline character at end of every file (Unix convention).
- Braces: always use braces/brackets even for single-line if/else (prevents off-by-one bugs).
- Quotes: pick single OR double quotes per language and stick to it consistently.

3. Function & Method Standards

- Functions should do ONE thing only (Single Responsibility applies here too).
- Keep functions short — aim for under 20–30 lines. Extract helpers if longer.

• Max 3–4 parameters per function. Use an object/dict for more arguments.

• Avoid side effects where possible — pure functions are easier to test.

• Return early ('guard clauses') to reduce nesting and improve readability.

• Name functions with a verb: getUser(), validateEmail(), sendNotification().

4. Comments & Documentation

Type	Best Practice
Inline comments	Explain WHY, not WHAT. Code shows what; comments explain intent.
JSDoc / docstrings	Document every public function: params, return type, description.
TODO / FIXME	Use TODO: for planned work, FIXME: for known bugs. Link to issue tracker.
Avoid dead code	Remove commented-out code. Git history preserves old code.
README	Every project needs a README: purpose, setup, usage, contributing.

5. Error Handling Standards

```
# Bad: silent failure
try:
result = risky_op()
except:
pass

# Good: specific exception, logged, re-raised or handled
try:
result = risky_op()
except ValueError as e:
logger.error(f'Validation failed: {e}')
raise HTTPException(status_code=400, detail=str(e))
```

6. Code Review Standards

• Every PR must be reviewed by at least one other developer before merging.

• PR size: keep under 400 lines changed — large PRs are hard to review effectively.

• All automated checks (tests, linting, type checks) must pass before review.

• Review for correctness, readability, security, and performance — in that order.

• Use constructive language: 'Consider using X because...' not 'This is wrong'.

- Authors should respond to every comment — agree, disagree with reason, or clarify.

7. Language-Specific Tooling

Language	Tools	Command
JavaScript/TS	ESLint + Prettier	npm run lint / npm run format
Python	flake8 / ruff + black	ruff check . / black .
Java	Checkstyle + SpotBugs	mvn checkstyle:check
Go	golint + gofmt	go fmt ./...
Rust	clippy + rustfmt	cargo clippy / cargo fmt
C#	StyleCop + Roslyn Analyzers	dotnet format

8. Git & Version Control Standards

- Commit often — small, atomic commits that represent one logical change.
- Use Conventional Commits format: feat:, fix:, docs:, refactor:, test:, chore:
- Branch naming: feature/login-page, fix/auth-bug, docs/api-readme
- Never commit secrets, passwords, or API keys — use .env files + .gitignore
- Squash WIP commits before merging to keep history clean.
- Tag releases with semantic versioning: v1.0.0, v1.1.0, v2.0.0

TOPIC 3

Authentication Types (OAuth, JWT, SSO & More)

Authentication verifies **who** a user is. Different mechanisms suit different use-cases — stateless APIs, web sessions, enterprise SSO, or machine-to-machine communication. Below is a comprehensive breakdown.

1. Overview Comparison

Method	State Type	Best For
Basic Auth	Stateful/Stateless	Simple apps, internal tools
API Key	Stateless	Service-to-service, public APIs
Session / Cookie	Stateful	Traditional web apps
JWT	Stateless	REST APIs, SPAs, mobile apps
OAuth 2.0	Stateless (token)	Third-party app access delegation
OpenID Connect	Stateless	Social login (Google, GitHub)
SAML 2.0	Stateful/browser	Enterprise SSO
LDAP / AD	Stateful	Corporate internal systems
mTLS	Stateless	Microservices, zero-trust networks
Biometric	Device-local	Mobile apps, device unlock
TOTP / MFA	Stateless (time)	Second-factor for any auth system
Passkeys (FIDO2)	Device-local	Passwordless future standard

A. Basic Authentication

The simplest form — username and password are Base64-encoded and sent in every request header. **Never use without HTTPS** as Base64 is trivially decodable.

```
Authorization: Basic dXNlcjpwYXNzd29yZA==
# base64('user:password') = dXNlcjpwYXNzd29yZA==
```

- Pros: Simple to implement, universally supported.
- Cons: Credentials sent every request, no expiry, HTTPS mandatory.
- Use case: Internal tools, quick prototypes, Swagger/API testing.

B. API Key Authentication

A random secret string issued to a client. Sent via header, query param, or request body. Has no user context — identifies an application, not a person.

```
# Header approach (preferred)
X-API-Key: sk-abc123xyz789
# OR
Authorization: Bearer sk-abc123xyz789
# Query param (less secure)
GET /api/data?api_key=sk-abc123xyz789
```

- Pros: Simple, stateless, easy to rotate.
- Cons: No user identity, no expiry by default, exposed if logged.
- Use case: OpenAI API, Google Maps API, Stripe, OpenRouter.

C. Session / Cookie Authentication

After login, the server creates a session and stores it server-side (DB/Redis). A session ID is sent to the client as an HTTP cookie. Every request sends the cookie and the server looks up the session.

```
Set-Cookie: sessionId=abc123; HttpOnly; Secure; SameSite=Strict
```

Flag	Purpose
HttpOnly	Cookie inaccessible to JavaScript — prevents XSS theft.
Secure	Cookie only sent over HTTPS.
SameSite	Strict / Lax — prevents CSRF attacks.
Max-Age	Expiry time in seconds.

- Pros: Easy revocation, user data stored server-side.
- Cons: Stateful — requires shared session store for horizontal scaling.
- Use case: Traditional MVC web apps (Django, Rails, Laravel).

D. JWT — JSON Web Token

A JWT is a self-contained, signed token carrying claims (data) about the user. The server doesn't need to store anything — it verifies the signature on each request. Stateless and perfect for REST APIs and microservices.

JWT Structure — 3 Parts Separated by Dots

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9 <-- Header (Base64)
.eyJ1c2VySWQiOiI0MiIsInJvbGUiOiJhZG1pbiIsImV4cCI6MTcwMDAwMH0 <-- Payload
.SflKxwRJSMeKKF2QT4fwpMeJf36POk6yJV_adQssw5c <-- Signature
```

Part	Contents

Header	Algorithm (HS256, RS256) and token type (JWT).
Payload	Claims: userId, email, role, exp (expiry), iat (issued at).
Signature	HMAC or RSA signature to verify the token wasn't tampered with.

JWT Flow

```
1. POST /login { email, password }
2. Server verifies credentials
3. Server returns:
{ "accessToken": "eyJ...", "refreshToken": "eyJ..." }
4. Client stores token (memory / localStorage / httpOnly cookie)
5. Every API call:
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
6. Server decodes + verifies signature — no DB lookup needed
```

Access Token vs Refresh Token

Token	Purpose & Lifespan
Access Token	Short-lived (15 min–1 hr). Used to call APIs. Stored in memory ideally.
Refresh Token	Long-lived (7–30 days). Used only to get new access token. Store httpOnly cookie.

• Pros: Stateless, scalable, self-contained, language-agnostic.

• Cons: Cannot be revoked before expiry (use short TTL + refresh token pattern).

• Signing Algorithms: HS256 (shared secret), RS256 (public/private key pair — preferred for multi-service).

E. OAuth 2.0 — Delegated Authorization

OAuth 2.0 lets users grant a third-party application limited access to their account on another service — WITHOUT sharing their password. Common example: 'Sign in with Google' or 'Connect your GitHub account'.

OAuth 2.0 Roles

Role	Description
Resource Owner	The user who owns the data and can grant access.
Client	Your application requesting access.
Authorization Server	Issues tokens after user grants consent (Google, GitHub, Auth0).
Resource Server	The API holding user data — validates access tokens.

Authorization Code Flow (Most Secure)

```
1. User clicks 'Login with Google'
2. App redirects to:
https://accounts.google.com/o/oauth2/auth
?client_id=APP_ID
&redirect;_uri=https://yourapp.com/callback
&scope;=email profile
&response;_type=code
&state;=RANDOM_STRING
3. User logs in & consents on Google's page
4. Google redirects back:
https://yourapp.com/callback?code=AUTH_CODE&state;=...
5. Your server exchanges code (POST to token endpoint):
{ code, client_id, client_secret, redirect_uri, grant_type }
6. Google returns: { access_token, refresh_token, expires_in }
7. Use access_token to call Google APIs on user's behalf
```

Grant Type	When to Use
Authorization Code	Standard web app flow. Requires backend server.
PKCE	Authorization Code without client_secret. For SPAs and mobile.
Client Credentials	Machine-to-machine. No user involved. Service authenticates itself.
Device Code	Smart TVs, CLIs. User enters a code on a separate device.

F. OpenID Connect (OIDC)

OIDC is an identity layer built on top of OAuth 2.0. While OAuth 2.0 handles **authorisation** (what can you access), OIDC adds **authentication** (who are you). It introduces the **ID Token** — a JWT containing user identity.

Feature	Description
ID Token	JWT with user claims: sub (user ID), name, email, picture.
UserInfo Endpoint	GET /userinfo returns user profile using the access token.
Scopes	openid (required), profile, email, address, phone.
Providers	Google, Microsoft (Azure AD), Apple, Okta, Auth0, Keycloak.

G. SAML 2.0 — Enterprise SSO

Security Assertion Markup Language — XML-based protocol for Single Sign-On in enterprise environments. A user logs in once to an Identity Provider (IdP) and gains access to multiple Service Providers (SPs) without re-authenticating.

Component	Description

Identity Provider (IdP)	Authenticates the user. Examples: Okta, Azure AD, ADFS, OneLogin.
Service Provider (SP)	The app the user wants to access. Trusts the IdP's assertion.
SAML Assertion	XML document signed by IdP confirming the user's identity + attributes.
SSO Flow	User → SP → Redirect to IdP → Login → IdP sends Assertion → SP grants access.

- Use case: Enterprise apps (Salesforce, Jira, AWS SSO, G Suite) with corporate credentials.
- SAML vs OIDC: SAML uses XML + browser redirects; OIDC uses JSON + API calls — OIDC is simpler for modern apps.

H. mTLS — Mutual TLS

In standard TLS, only the server presents a certificate. In **Mutual TLS**, both client and server present and verify certificates. This provides strong two-way authentication with no passwords or tokens required.

- Use case: Microservices communication, zero-trust networks, banking APIs, IoT devices.
- How: Each service has a client certificate signed by a trusted CA.
- Tools: Istio service mesh, NGINX, Envoy, Cloudflare mTLS.

I. MFA / TOTP — Multi-Factor Authentication

MFA adds an extra verification step after the primary credential check. TOTP (Time-based One-Time Password) generates a 6-digit code that changes every 30 seconds.

```
# TOTP formula (simplified)
TOTP = HMAC-SHA1(secret_key, floor(time() / 30))
# Output: 6-digit code valid for 30 seconds
```

- Apps: Google Authenticator, Authy, Microsoft Authenticator.
- Backup codes: Always provide 8-10 one-time backup codes for account recovery.
- TOTP standard: RFC 6238.
- SMS OTP: Weaker than TOTP (SIM-swap attacks) but better than no MFA.

J. Passkeys (FIDO2 / WebAuthn) — The Future

Passkeys replace passwords entirely using public-key cryptography tied to your device. No password is ever created, stored, or phished. The private key never leaves the device; the server only stores the public key.

- Login flow: biometric (FaceID / fingerprint) unlocks private key on device, signs challenge.

- Phishing-proof: keys are bound to the exact origin (domain), so fake sites can't steal them.
- Supported by: Apple, Google, Microsoft, major browsers (2023+).
- Standard: WebAuthn (W3C) + CTAP2 (FIDO Alliance).

TOPIC 4

Tokenizers in Different LLM Providers

A **tokenizer** converts raw text into a sequence of integer IDs (tokens) that a language model can process. Different providers use different tokenization schemes, vocabulary sizes, and algorithms — affecting cost, context window utilisation, and model behaviour.

1. What Is a Token?

- A token is roughly 3–4 characters of English text on average, or about 0.75 words.
- Rule of thumb: 1,000 tokens ≈ 750 words ≈ 3–4 paragraphs.
- Tokens can be whole words, sub-words, punctuation, spaces, or special markers.
- Non-English languages (Chinese, Arabic) often require MORE tokens per word.
- Code is token-dense — identifiers and syntax symbols each take tokens.
- Prices are charged per input + output token — understanding tokenization saves cost.

2. Common Tokenization Algorithms

Algorithm	Description
BPE — Byte Pair Encoding	Starts with characters; merges most-frequent pairs iteratively. Used by GPT series, LLaMA, Mistral.
WordPiece	Similar to BPE but maximises language-model likelihood. Used by BERT, DistilBERT, Gemma.
SentencePiece (Unigram)	Language-agnostic; works at byte level. Used by T5, LLaMA, Gemma, PaLM.
Tiktoken (cl100k_base)	OpenAI's fast BPE library in Rust. Used by GPT-4, GPT-3.5, embeddings.
Character-level	Every character is one token. Simple but long sequences. Rare in modern LLMs.

3. Tokenizers by Provider

OpenAI — Tiktoken

Model	Tokenizer	Details
GPT-4o, GPT-4, GPT-3.5	cl100k_base	100,277 vocab. ~4 chars/token for English.

GPT-4o mini	cl100k_base	Same tokenizer, cheaper model.
text-embedding-3	cl100k_base	Embeddings API uses same vocab.
GPT-2, Codex (legacy)	p50k_base	50,257 vocab. Older OpenAI models.
davinci, curie (legacy)	r50k_base	50,257 vocab. Completion models.

```
# Count tokens with tiktoken (Python)
import tiktoken
enc = tiktoken.encoding_for_model('gpt-4o')
tokens = enc.encode('Hello, how are you?')
print(len(tokens)) # 6
```

Anthropic (Claude) — Custom BPE

Model	Tokenizer	Notes
Claude 3.5 Sonnet/Haiku	Anthropic BPE	~200K context. Efficient multilingual.
Claude 3 Opus	Anthropic BPE	200K context window.
Claude 2	Anthropic BPE	100K context window.

Anthropic does not expose their tokenizer publicly. The Anthropic API returns usage.input_tokens and usage.output_tokens in each response. Approximate: 1 token ≈ 3.5–4 characters for English.

```
# Token count from API response
response = anthropic.messages.create(...)
print(response.usage.input_tokens) # e.g. 142
print(response.usage.output_tokens) # e.g. 87
```

Google (Gemini) — SentencePiece / WordPiece

Model	Tokenizer	Notes
Gemini 2.0 Flash	SentencePiece	1M context. Multimodal tokens included.
Gemini 1.5 Pro	SentencePiece	1M context window.
Gemini 1.0 Pro	SentencePiece	32K context.
PaLM 2 (legacy)	SentencePiece	Underlying Gemini predecessor.

```
# Count tokens via Google API
import google.generativeai as genai
model = genai.GenerativeModel('gemini-2.0-flash')
result = model.count_tokens('Hello, how are you?')
print(result.total_tokens) # 6
```

Meta — LLaMA / SentencePiece + Tiktoken

Model	Tokenizer	Notes
LLaMA 3.x	Tiktoken (cl100k)	128K+ context. Same vocab as GPT-4.
LLaMA 2	SentencePiece	32K vocab. 4K context.
Code LLaMA	SentencePiece	Code-optimised tokenizer.

Mistral AI — Tekken / SentencePiece

Model	Tokenizer	Notes
Mistral Large 2	Tekken (v3)	131K vocab. Better multilingual.
Mistral 7B	SentencePiece	32K vocab. v1 tokenizer.
Mixtral 8x22B	SentencePiece	32K vocab.

4. Tokens in Images & Audio (Multimodal)

Provider/Model	Multimodal Token Counting
OpenAI Vision	Each 512x512 image tile = 170 tokens. High-res: tiles + base = ~765–1K+ tokens.
Gemini	Images counted as flat token equivalent; video = frames * image tokens.
Claude Vision	Images up to 5MB; token cost varies by image size and resolution.
Whisper (Audio)	Audio transcribed first; resulting text tokens charged at standard rate.

5. Token Cost Comparison (Approximate — June 2026)

Model	Price (Input/Output)	Notes
GPT-4o	$2.50 / $10.00	Per million input/output tokens
GPT-4o mini	$0.15 / $0.60	Per million input/output tokens
Claude Sonnet 4.6	$3.00 / $15.00	Per million input/output tokens
Claude Haiku 4.5	$0.80 / $4.00	Per million input/output tokens
Gemini 2.0 Flash	$0.10 / $0.40	Per million input/output tokens
Llama 3 (via API)	~$0.20 / $0.30	Varies by provider (OpenRouter etc.)
Mistral Large	$2.00 / $6.00	Per million input/output tokens

6. Token Counting Tips

- Use provider SDKs or the Tokenizer Playground at platform.openai.com/tokenizer.
- Shorter system prompts = fewer input tokens = lower cost per request.
- Structured JSON prompts are often more token-efficient than verbose prose.
- Prompt caching (Claude, GPT-4o) can reduce repeated system-prompt costs by 80–90%.
- Streaming doesn't reduce token count — only improves perceived latency.
- Context window = max input + output tokens combined. Stay under the limit.

TOPIC 5

Postman API Practice

Postman is the most widely used API client for testing, documenting, and automating APIs. It provides a graphical interface to make HTTP requests, inspect responses, write test scripts, and manage environments — all without writing curl commands.

1. Postman Interface Overview

Feature	Description
Collections	Organised folders of API requests. Like a project for your API.
Requests	Individual HTTP calls (GET, POST, PUT, DELETE, PATCH).
Environments	Key-value variable sets (dev, staging, prod) to avoid hard-coding URLs.
Variables	Reusable values: {{baseUrl}}, {{token}}, {{userId}}.
Tests	JavaScript snippets that run after a response — assert status codes, body.
Pre-request Script	JavaScript that runs before the request — set dynamic variables.
Monitor	Scheduled runs of a collection to check API health.
Mock Server	Simulate API responses without a real backend — for frontend dev.
Documentation	Auto-generates docs from your collections, shareable via link.

2. Making Your First Request

Step-by-Step

- 1. Open Postman → Click New → Request.
- 2. Select HTTP method (GET, POST, etc.) from the dropdown.
- 3. Enter the URL: https://jsonplaceholder.typicode.com/posts/1
- 4. Click Send.
- 5. View Response: Status 200, JSON body, headers, response time.

Practice API — JSONPlaceholder (Free, No Auth Required)

Request	Action	Expected Response
GET /posts	List all posts	200 OK + array

GET /posts/1	Get post by ID	200 OK + single object
POST /posts	Create a new post	201 Created + new object
PUT /posts/1	Replace post completely	200 OK + updated object
PATCH /posts/1	Update specific fields	200 OK + patched object
DELETE /posts/1	Delete a post	200 OK + empty body

3. Request Components

A. Headers

Header	Value & Purpose
Content-Type	application/json — tell server the body format.
Authorization	Bearer TOKEN — send JWT or API key.
Accept	application/json — tell server what format you expect back.
X-API-Key	YOUR_API_KEY — custom API key header.

B. Request Body (for POST / PUT / PATCH)

```
// Raw JSON body – select 'raw' + 'JSON' in Postman
{
"title": "My First Post",
"body": "This is the content of my post.",
"userId": 1
}
```

C. Query Parameters

Add key-value pairs in the Params tab. Postman appends them to the URL automatically.

```
GET https://jsonplaceholder.typicode.com/posts?userId=1&_limit=5
# Params tab: userId=1, _limit=5
```

4. Environments & Variables

Environments prevent hard-coding URLs and tokens. Switch between dev/prod with one click.

Setup Example

```
Environment: 'Development'
Variables:
baseUrl = https://api.dev.myapp.com
token = eyJhbGciOiJIUzI1NiJ9...
userId = 42
# In requests, use double curly braces:
GET {{baseUrl}}/users/{{userId}}
```

```
Authorization: Bearer {{token}}
```

Variable Scopes (Priority Order)

Scope	Description
Global	Available in all collections and environments. Set sparingly.
Collection	Scoped to one collection. Good for collection-specific tokens.
Environment	Active environment variables. Switch environments to change.
Local	Set in pre-request scripts; cleared after request completes.

5. Writing Tests in Postman

Tests are JavaScript snippets in the 'Tests' tab. They run after every response.

```
// Test 1: Status code
pm.test('Status is 200', () => {
pm.response.to.have.status(200);
});
// Test 2: Response time
pm.test('Response under 500ms', () => {
pm.expect(pm.response.responseTime).to.be.below(500);
});
// Test 3: JSON body has property
pm.test('Body has id field', () => {
const json = pm.response.json();
pm.expect(json).to.have.property('id');
pm.expect(json.id).to.equal(1);
});
// Test 4: Save token from login response
const json = pm.response.json();
pm.environment.set('token', json.accessToken);
```

6. Pre-request Scripts

Run JavaScript BEFORE the request fires. Useful for generating timestamps, HMAC signatures, or refreshing tokens.

```
// Generate current Unix timestamp
pm.environment.set('timestamp', Date.now());
// Create a unique request ID
const uuid = require('uuid');
pm.environment.set('requestId', uuid.v4());
```

7. Authentication in Postman

Type	How to Set Up in Postman

No Auth	Open APIs — JSONPlaceholder, public endpoints.
API Key	Add key in Header or Query Param tab.
Bearer Token	Auth tab → Bearer Token → paste JWT.
Basic Auth	Auth tab → Basic Auth → username + password (auto Base64-encodes).
OAuth 2.0	Auth tab → OAuth 2.0 → fill in clientId, secret, token URL → Get Token.
AWS Signature	Auth tab → AWS Signature → Access Key, Secret Key, Region.

8. Collections & Automated Runs

- Organise requests into folders: Auth, Users, Products, Orders.
- Add tests to every request for automated validation.
- Run entire collection: Click '...' → Run Collection → Collection Runner.
- Set iteration count and data file (CSV/JSON) for data-driven testing.
- Newman: Postman's CLI runner — integrate into CI/CD pipelines.

```
# Run collection from command line with Newman
npm install -g newman
newman run MyCollection.json -e Development.json
newman run MyCollection.json --reporters cli,htmlextra
```

9. Postman Practice Exercises

Exercise	Task	Goal
Exercise 1	CRUD with JSONPlaceholder	GET, POST, PUT, DELETE on /posts
Exercise 2	Environment Variables	Set baseUrl, test in 3 environments
Exercise 3	Auth Flow	Login → save token → use in next request
Exercise 4	Write 5 Tests	Status, time, body fields, data types
Exercise 5	Mock Server	Create mock for /users endpoint
Exercise 6	Newman CI Run	Export collection, run with Newman CLI

10. Keyboard Shortcuts

Shortcut	Action	Note

Ctrl / Cmd + Enter	Send request	Most used shortcut
Ctrl / Cmd + S	Save request	Save after changes
Ctrl / Cmd + E	Manage environments	Quick env switch
Ctrl / Cmd + N	New request/tab	Open fresh tab
Ctrl / Cmd + /	Toggle sidebar	More workspace space
Ctrl / Cmd + L	Focus URL bar	Quick URL edit

End of Document